第三次基础实验 实验文档

Last Upd: 2026.10.13

- 第三次基础实验 实验文档
  - 01 实验目标与任务
    - 1 有理数四则运算模块
    - 2 知识图谱
  - 02 验收内容

01 实验目标与任务

1 有理数四则运算模块

目标是考查函数与模块。

编写一个模块 `rational.py` 实现有理数的四则运算。有理数的一般形式是 a/b，其中 a 是整数，b 是正整数，并且当 a 非零时 $|a|$ 和 b 的最大公约数是 1。

框架代码已经给出，待完成部分已经由 TODO 标记。程序中用一个列表 `[n, d]` 表示有理数，其中 `n` 表示分子， `d` 表示分母。 `reduce` 函数调用 `gcd` 函数进行约分。函数 `add` 、 `sub` 、 `mul` 和 `div` 分别进行加减乘除运算，运算的结果都需要约分，并且分母不出现负号。函数 `test_all_functions` 使用已知答案的数据对这些运算进行测试。

函数 `output` 按照示例的格式输出有理数，例如 `[-13,12]` 表示的有理数的输出结果是字符串 "-13/12"。

用户在命令行输入三个命令参数。 `--op` 表示运算符，可以是 "add"（加法）、"sub"（减法）、"mul"（乘法）或 "div"（除法）。 `--x` 和 `--y` 表示进行计算的两个有理数。有理数以字符串的形式输入，必须用圆括号括起，分子和分母之间用 "/" 分隔。例如有理数 "-20/-3" 对应的输入形式是 `(-20/-3)` ，用户输入的有理数可以在分母出现负号。函数 `get_rational` 从表示有理数的字符串中得到列表 `[n, d]` ，例如从字符串 "(-20/-3)" 得到 `[-20, -3]` 。

框架代码如下

```
# rational.py

"""
Module for performing arithmetic operations for rational numbers.

To run the module, user needs to supply three named parameters:
1. op stands for the operation:
    add for addition
    sub for subtraction
    mul for multiplication
    div for division
2. x stands for the first operand
3. y stands for the second operand

x and y must be enclosed in paired parentheses.

For example:

>>> run rational.py --op add --x (2/3) --y (-70/40)
-13/12
>>> run rational.py --op sub --x (-20/3) --y (120/470)
-976/141
>>> run rational.py --op mul --x (-6/19) --y (-114/18)
2/1
>>> run rational.py --op div --x (-6/19) --y (-114/-28)
-28/361
"""

import sys, math

### TODO 1: 补全测试函数以利用上述测试用例来测试结果 ###
def test_all_functions():

### TODO 1 结束 ###

def gcd(a, b):
    while a != b:
        if a > b:
            a -= b
        else:
            b -= a
    return a
```

```
### TODO 2: 补全 reduce 函数以对有理数 [n, d] 进行约分
def reduce(n, d):

### TODO 2 结束 ###

### TODO 3: 补全有理数的四则运算 ###
### ==============================

def add(x, y):

def sub(x, y):

def mul(x, y):

def div(x, y):

### ==============================
### TODO 3 结束 ###

### TODO 4: 补全 output 和 get_rational
### output: 输入为列表形式 x=[n,d]，直接打印在终端上；
### get_rational: 输入为用户输入的字符串，输出为其有理数形式 [n,d]
### ==============================

def output(x):

def get_rational(s):

### ==============================
### TODO 4 结束 ###

if __name__ == '__main__':
    if len(sys.argv) == 1:
        print(__doc__)
    elif len(sys.argv) == 2 and sys.argv[1] == '-h':
        print(__doc__)
    elif len(sys.argv) == 2 and sys.argv[1] == 'test':
        test_all_functions()
    else:
        import argparse
        parser = argparse.ArgumentParser()
        parser.add_argument('--op', type=str)
        parser.add_argument('--x', type=str)
```

```
    parser.add_argument('--y', type=str)
    args = parser.parse_args()
    op = args.op
    x = get_rational(args.x); y = get_rational(args.y)
    f = {'add':add, 'sub':sub, 'mul':mul, 'div':div}
    output(f[op](x, y))
```

解释一下底部的代码：

`if __name__ == '__main__'` 这句在实验课会讲到，实际上是用于区分当前被运行的脚本，和作为模块被导入的脚本。若指定了入口函数，则解释器只会执行该分支内的代码，而避免了 import 所导致的模块代码被错误执行的问题。

`sys` 是 python 的一个库，它用于存放与操作系统相关的函数。这里的 `sys.argv` 就是执行该命令时所传入的参数，和 C 语言中 `main()` 的 `int argc, char *argv[]` 一样，不过 python 并没有 `sys.argc`，而是使用列表获取长度的方法 `len(sys.argv)`。

`argparse` 是专门用于设计 python 脚本自定义参数的库，其支持自定义参数、默认值、解析读取与检查的操作。实际上在这里，就是对模块的一个调用，其从 `argparse` 模块中使用了 `ArgumentParser` 这个类，并将其实例化为变量 `parser`。

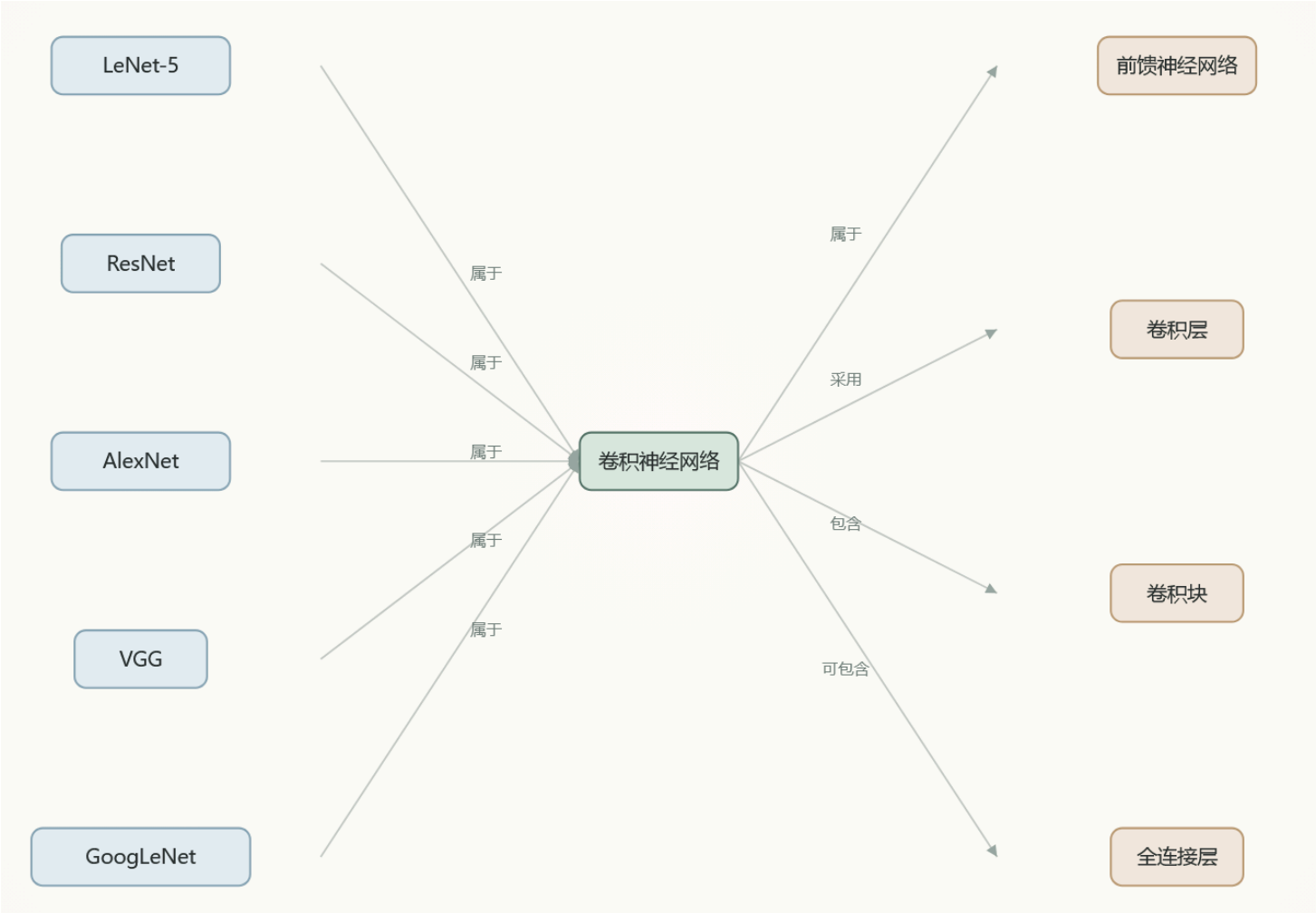
之后，为 `parser` 添加了自定义参数 `--op`（operation）、`--x`（左操作数）、`--y`（右操作数），并规定它们的类型为字符串。最后，通过调用类的方法函数 `parse_args` 获取用户输入的参数，作为其属性可直接访问。

最后，这里的 `f` 是一个近似函数指针的字典，其 Key 为运算类型 `add`、`sub`、`mul`、`div`，Value 为函数名，其先通过 `f[op]` 获取到用户指定的运算函数，之后将其作为函数并传入参数以调用 `f[op](x, y)`。最后调用自定义的 `output` 函数输出结果。

2 知识图谱

目标是考查函数、模块与包、lambda 函数、闭包、函数式编程。

知识图谱（Knowledge Graph）是一种以图结构组织和表示知识的方法，其核心思想是将知识中的概念、对象等抽象为实体（Entity），并利用关系（Relation）描述实体之间的联系。知识图谱通常采用 “主体—关系—客体” 的三元组（Triple）形式表示一条知识，例如（“反向传播算法”，“基于”，“链式法则”）和（“ResNet”，“采用”，“残差连接”）。其中，主体与客体构成图中的节点，关系构成连接节点的有向边。大量三元组相互连接，便形成了具有明确语义结构的知识网络。与传统的独立数据记录相比，知识图谱更加注重知识之间的关联，能够支持实体检索、关系查询、知识组织与关联分析等操作，广泛应用于搜索引擎、智能问答、推荐系统等领域。



本实验构建了一个简单的知识图谱系统，其读取 `chapter_triples.csv` 的既有三元组数据，并可对其进行简单的增、删、查操作（没有改。就是没有。）。其图谱可视化已经构建在 `view_graph.html` 中，上图是一个示例，大家可以自行打开体验。这个知识图谱是根据第三章的三节课件提取的，可供辅助复习之用。（这是直接把 pdf 丢给 ChatGPT6 Extra High 提取的，但我感觉这个提取效果并不好，应该需要更成熟的架构来提取。这样的 Agent 框架实际上并不少）

项目的结构如下

```
knowledge_graph_lab/
├── chapter_triples.csv        # 原始三元组数据
├── view_graph.html            # 可供体验的知识图谱页面
├── utils/
│   ├── __init__.py            # 待补全：完善 utils 包
│   └── load_data.py           # 已完成：加载数据
├── knowledge_graph/
│   ├── __init__.py            # 待补全：完善 knowledge_graph 包
│   └── knowledge_graph.py     # 待补全：知识图谱的六个核心操作函数
├── exercise.py                # 待补全：简单的复合测试用例
└── test_public.py             # 已完成：基本的检测用例
```

项目的工作流大概是： `utils` 中的 `load_data.py` 提供了抽取数据并构建知识图谱的函数，而 `knowledge_graph.py` 则进一步提供了对它的操作。它们都只提供了函数，内部并没有持久性存储变量或数据。

`test_public.py` 是针对知识图谱自身的测试，用来测试其核心代码是否被正确补全。 `exercise.py` 则提供了一些进阶的复合操作，它们由 `knowledge_graph.py` 中的核心函数组合而成，能够实现一些更复杂的功能。

这种类似编写库然后调用的方法实际上在 C 语言中十分常见，只不过在 C 中，这些库的代码是以静态库 `.lib` 、动态库 `.so` 或头文件 `.h` 的方式出现的，这实际体现了面向过程编程的特点。

后续的实验课中，将会引入类的概念，届时采用面向对象的思路，这种项目组织结构会显得更加自然。

02 验收内容

对于实验 1，其代码能正确执行并通过自己编写的测试。

对于实验 2，其代码能正确执行，并通过 `exercise.py` 和 `test_public.py` 的测试。